Instrumented & Monitored Cloud Service

Presented by :Rizwan

Made with GAMMA

The Problem This Project Solves

The World Without Observability

At 2am, your phone buzzes: "Orders page broken". You SSH to the server. You run `ps aux`. You grep some logs. You stare at the screen. You don't know:

- When did it start breaking?
- Is it getting worse or better?
- Is it the database, the API, the frontend, or the network?
- How many users are affected?
- What changed?

You're guessing. Every minute of guessing costs money and user trust.

The World With Observability

Same 2am alert. You open your phone, pull up CloudWatch dashboard. In 30 seconds:

1. Error rate spiked from 0% to 18% at exactly 1:47am
2. P95 latency is normal (500ms errors, not slow errors — points to code bug)
3. Request rate is normal (it's not a traffic attack)
4. Memory and CPU are normal (not resource exhaustion)
5. The spike correlates with a deploy at 1:46am (someone pushed broken code)

You rollback in 5 minutes. Incident resolved. Users barely noticed.

Made with GAMMA

What Was Built

A simple Order Processing REST API as the vehicle for demonstrating production-grade observability. The application deliberately kept simple so the observability layer is the focus.

Application API

POST /orders, GET /orders/:id, GET /orders, DELETE /orders/:id, GET /health, GET /metrics, POST /chaos endpoints for simulating failures

Structured Logging

Every request produces structured JSON log lines with correlation IDs, event names, and relevant fields flowing to CloudWatch Logs

Custom Metrics

6 aggregate metrics published every 60 seconds plus per-event metrics on each order action plus system metrics from CloudWatch Agent

CloudWatch Dashboard

10 widgets organized by Golden Signals showing entire system state in one place

Tiered Alarms

7 alarms with Warning and Critical tiers sending email notifications via SNS

Made with GAMMA

How Everything Connects

Every HTTP request flows through the complete observability stack, from user to metrics storage to alerting.

User Request

HTTP POST to /orders with JSON body containing customer_id and items

Flask Middleware

Generates correlation ID, increments counters, logs request_started event

Route Handler

Validates input, generates order_id, stores in memory, publishes per-event metrics, logs order_created

Background Thread

Every 60 seconds: computes ErrorRate, P95LatencyMs, aggregates metrics, publishes to CloudWatch

CloudWatch Storage

Stores metrics with dimensions, evaluates alarm conditions every period

Alarm & SNS

Threshold breached → ALARM state → SNS publishes → Email notification sent

Made with GAMMA

The Full Request Lifecycle

Every HTTP request to the Order API goes through these exact steps:

01

TCP Connection

Established to EC2:5000

02

Gunicorn Accept

Worker accepts connection (Round-robin across 2 workers, each with 4 threads = 8 concurrent max)

03

Flask Parse

Parses HTTP request into the 'request' proxy object

04

before_request

Records start time, assigns correlation ID, increments counters, logs request_started

05

Route Handler

Validates input, executes business logic, publishes metrics, logs business event, returns response

06

jsonify

Flask calls jsonify on return value → JSON response object

07

after_request

Calculates duration, appends to latencies list, increments success/error, decrements sessions, logs completion, adds correlation header

08

Response Sent

Gunicorn sends HTTP response to client with X-Correlation-ID header

Exception Handling: If an unhandled exception occurs, Flask's errorhandler fires instead of after_request — it logs the error with traceback and publishes an UnhandledErrors metric.

Made with GAMMA

The Full Metrics Lifecycle

Immediate Metrics

Handler calls publish_metrics_batch() → boto3.put_metric_data() makes HTTPS call to CloudWatch API → CloudWatch stores data point with timestamp

- Namespace: OrderAPI/Production
- MetricName: OrdersCreated, OrderValue
- Dimensions: Service=order-api, Environment=production

Aggregate Metrics

Background thread wakes every 60 seconds → Acquires lock, copies and resets counters → Computes ErrorRate, P95LatencyMs, AvgOrderValue → publish_metrics_batch() → CloudWatch

System Metrics

CloudWatch Agent reads /proc/stat → CPU metrics, /proc/meminfo → Memory metrics, /proc/diskstats → Disk metrics → Sends to CloudWatch via HTTPS

- Namespace: OrderAPI/System
- Metrics: cpu_usage_user, mem_used_percent, used_percent

Made with GAMMA

The Full Alerting Lifecycle

CloudWatch evaluates metrics every period (60 seconds) and triggers state transitions based on threshold breaches.

1

Period 1

ErrorRate = 0.3% → below 1% → data point = OK

2

Period 2

ErrorRate = 1.8% → above 1% → data point = BREACH

3

Period 3

ErrorRate = 2.1% → above 1% → data point = BREACH

4

ALARM State

2 out of 3 periods breached ✓ → Alarm state transitions to ALARM → SNS notification published

5

Recovery

3 consecutive OK periods → state transitions to OK → RESOLVED notification sent

Made with GAMMA

Incident Investigation Playbook

When an alarm fires, follow this exact process to identify and resolve the issue:

Identify the Signal

Open CloudWatch Dashboard. Find the red/elevated widget. Answer: Is this Rate, Errors, Latency, or Saturation?

Apply RED Method

Rate: Open Request Rate widget. Errors: Open Error Rate and Errors by Type widgets. Duration: Open P95 Latency widget. Determine pattern.

Apply USE Method

If Saturation elevated: CPU high + latency high = CPU bottleneck. Memory high = memory leak. Disk high = check logs growing. iowait high = disk I/O bottleneck.

Query Logs for Root Cause

Open CloudWatch Logs Insights: find error patterns, identify exact errors, trace specific requests using correlation_id from alert.

Identify Fix

Based on evidence: errors at deploy time = rollback. Errors + high CPU = scale up. Errors + high memory = restart, find leak. Silent failures = check business logic.

Document Incident

Write entry in INCIDENTS.md using template: timeline with timestamps, RED/USE analysis, root cause, fix applied, lessons learned.

Made with GAMMA

Design Decisions and Trade-offs

Key architectural choices and why they matter:

Direct boto3 vs OpenTelemetry

Chose direct boto3 + structlog for zero external infrastructure, explicit code, lower learning curve. In production: use OpenTelemetry for vendor neutrality.

One Namespace vs Per-Service

Using OrderAPI/Production for all metrics. Trade-off: must use dimensions to filter by service. Multiple namespaces better at scale for cross-service dashboards.

In-Memory Storage

No database setup, zero connection pooling, zero DB monitoring. Focus is observability, not application complexity. In production: use RDS or DynamoDB.

60-Second Publication

CloudWatch standard-resolution metrics have 1-minute granularity. Publishing more frequently wastes API calls — data gets aggregated to 1-minute buckets anyway.

P95 vs P99/P50

P95 is industry standard — captures "long tail" without being overly sensitive to rare outliers. P99 can cause false alarms on low-traffic services.

Made with GAMMA

Production vs Demo Differences

What would be different in real production environments:

Layer	Demo Implementation	Production Implementation
Application	In-memory dict storage Single EC2 instance Manual deployment No authentication	RDS PostgreSQL or DynamoDB Auto Scaling Group behind ALB CI/CD pipeline JWT/OAuth2 with API Gateway
Observability	Direct boto3 metrics stdout → CloudWatch Agent Correlation IDs only Static alarm thresholds Email notifications	OpenTelemetry SDK + Collector Fluentd/Fluent Bit aggregation Distributed tracing with X-Ray Anomaly detection alarms PagerDuty + Slack + email
Infrastructure	Single region No CDN No VPC configuration Root IAM permissions No backup/disaster recovery	Multi-region with Route 53 failover CloudFront for static assets Private subnets, NAT gateway, security groups Least-privilege IAM roles RDS snapshots, cross-region replication

Made with GAMMA